(a)
Due to the rapid nature of the project, we needed to ensure that the testing process was as efficient as possible. This meant using as many automated tests as possible and making use of LibGDX's headless backend in order to speed up the testing time by not having to fully render the graphics of the game. We also checked to ensure that there was as little overlap between tests as possible.

Another thing that we did was ensure that tests were being written as the code was being written. This meant going back through the previous team's code to ensure that the test coverage and pass rate was sufficient, writing any additional tests necessary, and only then could we start adding new code. This made it easier to isolate faults and fix them before they affected other sections of code. Then, when writing new code, we made sure to follow object-oriented principles to allow for many smaller unit tests to be written so that sections of code could be tested in isolation as the project progressed and then be tested together with a small number of integration tests and an even smaller number of end-to-end tests, avoiding the testing antipatterns.

As getting 100% test coverage would be impossible and inefficient, we needed a way to ensure that our testing was sufficient therefore we decided to use a combination of test coverage, boundary checks, and a traceability matrix. The test coverage was useful as a rough guide, with the aim to get roughly 90% coverage. The boundary check procedure made sure that the coverage was of high quality by ensuring that all relevant sections of code were tested with normal, boundary, and erroneous data and that they reacted appropriately. Finally, the traceability matrix acted as another way to ensure that testing was sufficient and appropriate by making sure that every test is linked to at least one requirement and that there aren't too many tests for each requirement (indicating overlap and therefore inefficiency).